

# Perché un modello predittivo deve essere testato su dati indipendenti

---

## Obiettivo

Quando costruiamo un modello di Machine Learning, non ci interessa soltanto sapere quanto bene riesce a descrivere o prevedere i dati sui quali è stato addestrato.

La vera domanda è:

**Quanto bene funzionerà su nuovi casi che non ha mai visto?**

Questa capacità si chiama **generalizzazione** ed è uno degli obiettivi fondamentali del Machine Learning predittivo.

---

## 1. Un esempio bancario

Supponiamo di voler costruire un modello per prevedere il **rischio di default di un cliente**.

Disponiamo di 100.000 clienti storici per i quali conosciamo, ad esempio:

- reddito;
- anzianità lavorativa;
- esposizione complessiva;
- utilizzo delle carte;
- numero di ritardi nei pagamenti;
- rapporto rata/reddito;
- altri indicatori comportamentali e finanziari.

Per questi stessi clienti sappiamo anche se, successivamente, si è verificato un default.

Il nostro dataset contiene quindi:

- le **feature**, cioè le informazioni disponibili sul cliente;
- la **variabile target**, cioè l'esito che vogliamo prevedere.

Se utilizziamo tutti i 100.000 clienti per addestrare il modello e poi misuriamo le prestazioni sugli stessi 100.000 clienti, stiamo in realtà chiedendo:

**Quanto sei bravo sui casi che hai già studiato?**

Ma la banca vuole sapere qualcosa di diverso:

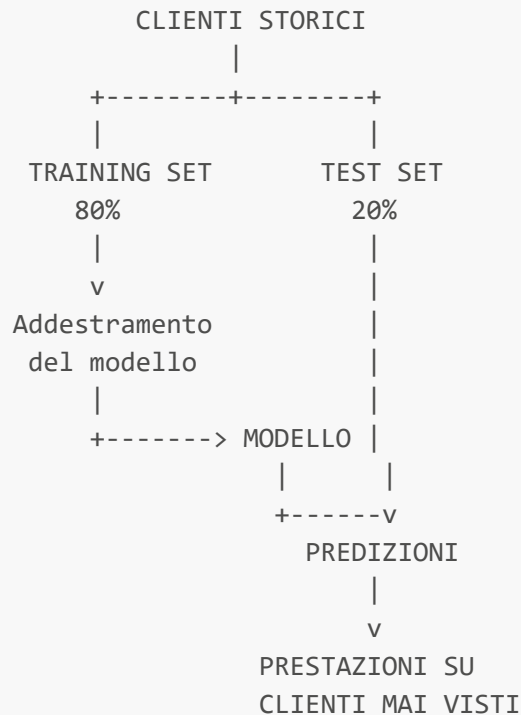
**Quanto sarai bravo sul prossimo cliente che non hai mai visto?**

Per rispondere a questa seconda domanda dobbiamo valutare il modello su dati che non sono stati utilizzati per addestrarlo.

---

## 2. Training set e test set

Un approccio semplice consiste nel dividere i dati storici in due gruppi indipendenti.



Il **training set** serve per:

**imparare**

Il **test set** serve per:

**verificare se ciò che è stato imparato funziona anche su casi nuovi**

Le percentuali 80% / 20% sono soltanto un esempio. Si possono usare altre suddivisioni, come 70% / 30%, oppure tecniche come la cross-validation.

Il principio importante non è la percentuale scelta.

Il principio importante è che:

**le osservazioni usate per valutare il modello non devono essere le stesse utilizzate per addestrarlo.**

## 3. L'analogia dell'esame

Un modo intuitivo per comprendere il problema è immaginare uno studente.

Supponiamo di preparare un candidato fornendogli:

- 1.000 domande;
- le relative risposte corrette.

Dopo un periodo di studio decidiamo di valutarlo.

Se l'esame utilizza **esattamente le stesse 1.000 domande**, il candidato potrebbe ottenere il 100% delle risposte corrette.

Ma possiamo concludere che conosce davvero la materia?

Non necessariamente.

Potrebbe semplicemente avere **memorizzato le risposte**.

Un esame serio utilizza quindi domande nuove.

Nel Machine Learning accade la stessa cosa:

```
TRAINING DATA
  |
  v
MODELLO
  |
  v
"Ha imparato le regole?"
  oppure
"Ha memorizzato gli esempi?"
  |
  v
TEST DATA MAI VISTI
  |
  v
Lo scopriamo qui
```

Il **test set** svolge quindi un ruolo analogo alle domande nuove di un esame.

---

## 4. Overfitting

Questa distinzione introduce uno dei concetti più importanti del Machine Learning:

### overfitting

Un modello è in overfitting quando si adatta molto bene ai dati di training, ma non riesce a generalizzare altrettanto bene su dati nuovi.

In forma schematica:

```
Prestazioni sul training:  MOLTO ALTE
Prestazioni sul test:      MOLTO PIÙ BASSE
                          |
                          v
                    possibile OVERFITTING
```

Un modello troppo complesso può imparare non soltanto le relazioni realmente presenti nei dati, ma anche:

- rumore;
- anomalie;
- peculiarità del campione;
- combinazioni casuali che non si ripresenteranno in futuro.

Per questo una performance molto elevata sul training set **non è di per sé una garanzia di qualità**.

---

## 5. In-sample e out-of-sample

Possiamo distinguere tra due tipi di valutazione.

### Performance in-sample

È la performance misurata sui dati utilizzati per costruire il modello.

Risponde soprattutto alla domanda:

**Quanto bene il modello ha imparato i dati che conosce?**

### Performance out-of-sample

È la performance misurata su dati che il modello non ha utilizzato durante l'addestramento.

Risponde alla domanda più importante:

**Quanto bene il modello generalizza a nuovi casi?**

Per un modello predittivo destinato a essere utilizzato in produzione, la seconda misura è quella più importante.

---

## 6. Il vero obiettivo: generalizzare

Un algoritmo predittivo non viene costruito per prevedere il passato.

Viene costruito per effettuare previsioni su nuove osservazioni.

In ambito bancario, ad esempio:

- nuovi clienti;
- nuove richieste di credito;
- nuove transazioni;
- nuovi eventi di pagamento;
- nuovi comportamenti osservati nel tempo.

Per questo possiamo riassumere il concetto in tre frasi.

**Training measures learning.**

**Testing measures generalization.**

E, in termini ancora più intuitivi:

**Il test set è una simulazione del futuro.**

## 7. Come mostrarlo in Orange

Il concetto può essere verificato direttamente con Orange.

### Approccio concettualmente scorretto

Si costruisce un modello e lo si valuta sugli stessi dati utilizzati per addestrarlo.

```
DATA
|
v
MODELLO
|
v
PREDIZIONI SUGLI STESSI DATI
```

La misura ottenuta può essere eccessivamente ottimistica.

Con algoritmi molto flessibili, ad esempio:

- Decision Tree molto profondo;
- k-NN con configurazioni particolarmente aderenti ai dati;
- modelli con elevata complessità;

la differenza può essere evidente.

## 8. Hold-out test con Orange

Una modalità più corretta consiste nel separare i dati di training da quelli di test.

Un possibile workflow concettuale è:

```
Data
|
v
Data Sampler
| \
|  \
|   \ +-----> Test Data
|    \
|     \ +-----> Training Data
|      \
|       \
|        \
|         \
+-----> Training Data
          |
          v
        Model
```



Il modello viene quindi costruito sui dati di training e valutato sui dati che non ha utilizzato per imparare.

In questo modo la misura ottenuta è molto più vicina alla domanda reale:

**Come si comporterà il modello su clienti che non ha mai visto?**

## 9. Cross-validation

Quando il dataset non è enorme, una singola suddivisione training/test può dipendere in parte dal caso.

La **cross-validation** consente di utilizzare più volte i dati, mantenendo però separati, in ogni iterazione, i casi utilizzati per il training e quelli utilizzati per il test.

Ad esempio, con una 5-fold cross-validation:

|        |       |       |       |       |       |
|--------|-------|-------|-------|-------|-------|
| Fold 1 | TEST  | TRAIN | TRAIN | TRAIN | TRAIN |
| Fold 2 | TRAIN | TEST  | TRAIN | TRAIN | TRAIN |
| Fold 3 | TRAIN | TRAIN | TEST  | TRAIN | TRAIN |
| Fold 4 | TRAIN | TRAIN | TRAIN | TEST  | TRAIN |
| Fold 5 | TRAIN | TRAIN | TRAIN | TRAIN | TEST  |

Ogni osservazione viene quindi usata:

- più volte per l'addestramento;
- una volta per la valutazione.

Le prestazioni finali vengono aggregate sulle diverse iterazioni.

In Orange, il widget **Test & Score** consente di utilizzare direttamente la cross-validation.

## 10. Un punto particolarmente importante in banca: il tempo

In molti problemi bancari una divisione casuale training/test non è sempre la prova più realistica.

Supponiamo, ad esempio, di voler costruire un modello utilizzando dati raccolti tra il 2019 e il 2024.

Una valutazione particolarmente significativa può essere:

|          |      |      |      |      |      |      |      |      |  |      |
|----------|------|------|------|------|------|------|------|------|--|------|
| 2019     | ---- | 2020 | ---- | 2021 | ---- | 2022 | ---- | 2023 |  | 2024 |
| TRAINING |      |      |      |      |      |      |      |      |  | TEST |

Il modello viene addestrato sui dati disponibili fino al 2023 e testato sui clienti del 2024.

La domanda diventa:

**Se avessimo costruito il modello alla fine del 2023, come si sarebbe comportato sui clienti osservati nel 2024?**

Questa impostazione simula molto bene il reale utilizzo operativo del modello.

---

## 11. Perché il test temporale può essere più realistico

Il mondo cambia.

Nel settore bancario possono cambiare:

- condizioni macroeconomiche;
- tassi di interesse;
- inflazione;
- comportamento dei clienti;
- politiche commerciali;
- criteri di concessione del credito;
- composizione della clientela;
- modalità di pagamento;
- incidenza delle frodi;
- regolamentazione.

Di conseguenza, le relazioni presenti nei dati storici possono modificarsi nel tempo.

Questo introduce concetti come:

### Population drift

La distribuzione delle caratteristiche della popolazione cambia nel tempo.

Ad esempio, i clienti del 2026 potrebbero avere caratteristiche diverse rispetto ai clienti del 2021.

### Concept drift

Può cambiare la relazione tra le feature e la variabile target.

Ad esempio, un certo livello di indebitamento potrebbe avere un significato diverso in periodi caratterizzati da tassi di interesse molto differenti.

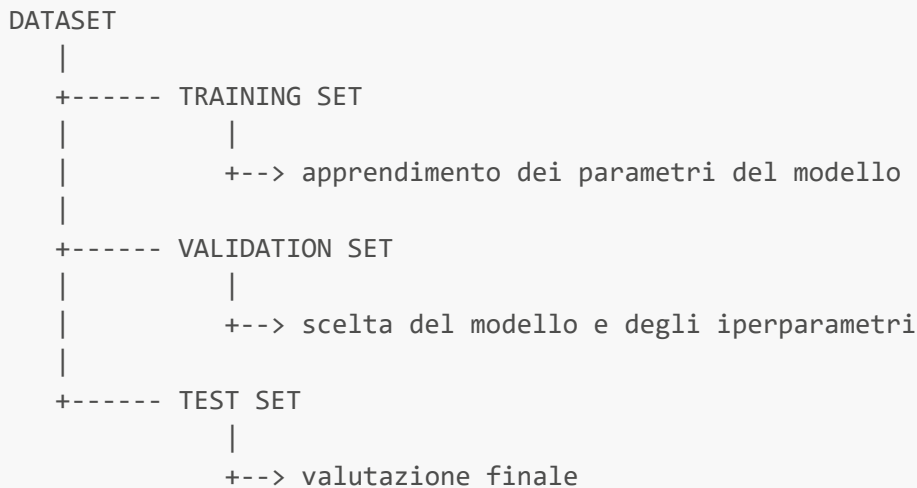
Per questo, nei sistemi reali, non basta valutare il modello una sola volta.

Le performance devono essere monitorate nel tempo.

---

## 12. Training, validation e test

Nei progetti più strutturati si utilizzano spesso tre insiemi distinti:



È importante comprendere il motivo di questa ulteriore separazione.

Se proviamo molte configurazioni e scegliamo quella che funziona meglio sul test set, stiamo indirettamente utilizzando il test set per prendere decisioni sul modello.

A quel punto il test set non è più completamente indipendente.

Per questo:

- il **training set** serve per imparare;
- il **validation set** serve per scegliere;
- il **test set** serve per la valutazione finale.

La cross-validation può spesso svolgere il ruolo del validation set durante la fase di sviluppo.

---

## 13. Attenzione al data leakage

La separazione tra training e test deve riguardare non soltanto l'algoritmo finale, ma l'intera pipeline di Machine Learning.

Il test set non dovrebbe influenzare operazioni come:

- selezione delle feature;
- normalizzazione;
- standardizzazione;
- imputazione dei valori mancanti;
- scelta degli iperparametri;
- selezione del modello.

Se utilizziamo informazioni provenienti dal test set durante queste operazioni, introduciamo **data leakage**.

Il risultato può essere una stima delle performance eccessivamente ottimistica.

Il principio generale è:

**Tutto ciò che viene "imparato" dai dati deve essere appreso utilizzando soltanto il training set.**

Solo successivamente la trasformazione già appresa viene applicata al test set.

---

## 14. La domanda corretta da fare a un modello

Una performance elevata sui dati storici non è sufficiente.

La domanda sbagliata è:

**Quanto bene il modello riproduce i dati sui quali è stato costruito?**

La domanda corretta è:

**Quanto bene il modello riesce a prevedere casi indipendenti e mai osservati durante l'addestramento?**

Questa è la differenza tra **adattamento ai dati** e **capacità predittiva**.

---

## 15. Messaggio finale

Un modello predittivo deve essere valutato su dati indipendenti perché il suo vero compito non è ricordare il passato, ma **generalizzare dal passato al futuro**.

Possiamo riassumere tutto in quattro idee:

1. **Il training set serve per imparare.**
2. **Il test set serve per verificare la generalizzazione.**
3. **Una buona performance sul training non garantisce una buona performance su nuovi dati.**
4. **Il test set rappresenta, per quanto possibile, una simulazione dei casi futuri che il modello incontrerà in produzione.**

La frase da ricordare è:

**A predictive model should not be judged on its ability to predict the past it has already seen, but on its ability to predict unseen cases.**

Oppure, in forma ancora più sintetica:

**Training measures learning. Testing measures generalization.**

E, pensando all'utilizzo reale in banca:

**Il test set è una simulazione del futuro.**